COMSATS University Islamabad,

Sahiwal Campus

Smart Defence Rover

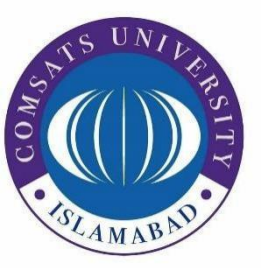
By

Abdullah Ahsan	**CIIT/SP22-BCS-014/SWL**
Wasif Ali	**CIIT/SP22-BCS-120/SWL**
Muhammad Husnain	**CIIT/SP22-BCS-138/SWL**

Supervisor

Ms. Asma Iqbal

Bachelor of Science in Computer Science (2022-2026)



COMSATS University Islamabad,

Sahiwal Campus.

Smart Defence Rover

A Project Presented To

COMSATS University Islamabad, Sahiwal Campus

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Abdullah Ahsan	**CIIT/SP22-BCS-014/SWL**
Wasif Ali	**CIIT/SP22-BCS-120/SWL**
Muhammad Husnain	**CIIT/SP22-BCS-138/SWL**

DECLARATION

We hereby declare that the project titled **"Smart Defence Rover"** including its software, hardware integration, and the accompanying report is entirely our original work, developed through our own knowledge, effort, and research. No part of this project has been copied from any source or reproduced from the work of others in a manner that violates academic honesty. We fully accept responsibility for the authenticity of this work and will abide by all consequences if any portion is found to be plagiarized or submitted elsewhere. Furthermore, we confirm that this project has not been presented for any degree, certificate, or qualification at this or any other institution.

Abdullah Ahsan **Wasif Ali** **Muhammad Husnain**

CERTIFICATE OF APPROVAL

This is to certify that the final year project of BS(CS) **"Smart Defence Rover"** was developed by Abdullah Ahsan **(CIIT/SP22-BCS-014/SWL),** Wasif Ali **(CIIT/SP22-BCS-120/SWL)** and Muhammad Husnain **(CIIT/SP22-BCS-138/SWL)** under the supervision of "**Ms. Asma Iqbal**" and co supervisor "**Dr.Yawar Abbas Abid**" and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

`

Supervisor

Ms. Asma Iqbal

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

`

External Examiner

Dr. Sajid Ali

Associate Professor

Department of Computer Science

University of Education Multan

Head of Department

Dr. Zafar Iqbal Roy

Associate Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

In today's rapidly changing environment, safety threats such as sudden fire incidents and unauthorized intrusions require faster, smarter, and more reliable solutions than traditional alarm systems and manual surveillance can provide. Smoke detectors and CCTV cameras offer basic monitoring, but they lack the ability to respond instantly or take autonomous action when an emergency occurs. The **Smart Defence Rover** is designed to overcome these limitations by providing real-time fire detection, and automatic on-ground response through artificial intelligence and robotics.

The rover uses advanced technologies such as **OpenCV for flame detection**, allowing it to analyze live video and detect threats accurately. Once a fire, the system captures a snapshot, and sends an email alert to the user. Simultaneously, it activates its **water pump** to suppress flames or its **buzzer alarm** to signal Fire Presence. These automated actions make the rover a fast and efficient safety unit capable of reacting without human involvement.

The system is built with a modular architecture where each component plays a critical role. The **Raspberry Pi** handles AI processing and decision-making, while the **Arduino** controls actuators such as motors, the pump, and the buzzer. The integrated **Camera Module** supplies continuous visual input. Together, these modules create a coordinated and reliable platform that maintains logs, stores snapshots, and ensures seamless communication.

Overall, the Smart Defence Rover is a compact yet powerful robotic solution that integrates AI-based detection, autonomous firefighting, and real-time email reporting. By combining computer vision with embedded systems and mobile robotics, it enhances emergency response and reduces risks to human life, offering a smarter and more efficient approach to safety in modern environments.

Acknowledgement

All praise is to the almighty Allah, who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **"Ms. Asma Iqbal".** Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

We are also thankful to our parents and family, who have been a constant source of encouragement for us and have brought us the values of honesty & hard work.

Abdullah Ahsan **Wasif Ali** **Muhammad Husnain**

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
CV	Computer Vision
RPI	Raspberry Pi
GPIO	General Purpose Input Output
YOLO	You Only Look Once (Object Detection Model)
ML	Machine Learning
SMTP	Simple Mail Transfer Protocol
GPS	Global Positioning System
UART	Universal Asynchronous Receiver-Transmitter
DC	Direct Current
RPM	Revolutions Per Minute
FPS	Frames Per Second
IDE	Integrated Development Environment
USB	Universal Serial Bus
PWM	Pulse Width Modulation
IR	Infrared
LED	Light Emitting Diode
IC	Integrated Circuit
API	Application Programming Interface
SDD	Software Design Description
SRS	Software Requirement Specification
GUI	Graphical User Interface

Table of Contents

List of Tables

List of Figures

Chapter 1:

Introduction

1 Introduction:

Over the past decade, the need for intelligent autonomous systems in the fields of surveillance, safety, emergency response, and defense has increased significantly. Traditional firefighting and security systems rely heavily on human intervention, which introduces delays, risks to human life, and limited coverage. Fire outbreaks, criminal intrusions, and emergency hazards require rapid identification and immediate action something manual systems often fail to provide.

To address these limitations, the Smart Defence Rover is developed as a fully automated, AI-driven robotic vehicle capable of performing emergency detection and response tasks. It combines Computer Vision, Machine Learning, Robotics, IoT, and Embedded Systems to create a high-accuracy, real-time detection and reaction platform.

The rover autonomously detects:

- Fire (using OpenCV color segmentation / ML models)
- Emergency conditions requiring instant response

Upon detection, it automatically:

- Captures a snapshot of the scene
- Sends an alert email to the registered user
- Activates a water pump to extinguish fire
- Positions the camera

This robotic system reduces human risk, increases response time, and provides continuous 24/7 monitoring.

1.1 Brief Overview

The Smart Defence Rover functions as an intelligent autonomous mobile unit that operates in indoor and outdoor environments. It carries out surveillance, detection, and response tasks with minimal human intervention.

Key Functions:

Fire Detection & Extinguishing

- Detects flame patterns via camera
- Auto-activates water pump via relay
- Directs nozzle using servo motor mechanism

Snapshot + Email Alerts

- Sends image + fire
- Uses Raspberry Pi's SMTP client

Rover Mobility

- Controlled via Arduino Uno + dual L298N motor drivers
- Can move in all directions
- Stable 4-wheel chassis for rugged terrain

Modular Design

- Easy upgradable hardware
- Software-based logic for flexible enhancements

The system integrates both hardware intelligence (sensors, relays, motors) and software intelligence (AI models, signal processing, and automation scripts).

1.2 Relevance to Course Modules

This project reflects your academic background as a Computer Science student. The following subjects directly contributed to building this rover:

Programming Fundamentals & OOP

- Python code for fire detection
- Arduino C/C++ for motor & sensor control
- Modular coding practices

Data Structures & Algorithms

- Frame-by-frame processing
- Efficient image filtering
- Real-time decision making

Artificial Intelligence

- Fire classification using ML/CNN models

Computer Networks

- Email communication
- Data transfer + remote monitoring

Embedded Systems

- Raspberry Pi GPIO interfacing
- Arduino inputs/outputs
- Relay & sensor integration

Operating Systems

- Linux (Raspberry Pi OS)
- Multithreading for real-time tasks

Human Computer Interaction (HCI)

- Simple operator interface
- Visual indicators & alarms

Software Engineering

- Requirement gathering
- Modular design
- Use case modelling
- Testing methods

Final Year Project Methodology

- SDLC
- Document preparation
- Prototyping & iterative testing

This project is a practical application of almost every major subject studied during your degree.

1.3 Project Background

Fire hazards, unauthorized intrusions, and emergency incidents pose serious risks across residential, commercial, industrial, and agricultural environments. These threats can lead to severe property damage, financial losses, and in many cases, endanger human lives. Despite advancements in safety equipment, traditional systems still operate with significant limitations. Most commonly deployed solutions include:

- Smoke and heat detectors
- CCTV surveillance cameras
- Security guards and manual patrolling

- Conventional alarm systems

Although these tools provide basic monitoring and initial alerts, they do not offer real-time autonomous action, nor can they actively control or mitigate a hazardous situation. Their major shortcomings include:

- Lack of automatic response: Systems can detect a problem but rely on humans to take action.
- No fire extinguishing capability: Smoke detectors can only beep they cannot suppress fire.
- Inability to identify human presence: CCTV cannot differentiate between normal activity and a threat without human monitoring.
- No evidence capturing mechanism: Many systems fail to automatically record or log events with proof.
- No integrated GPS tracking: Most setups cannot report their exact location during emergencies.
- Immobility: Existing systems remain fixed in one spot, limiting their coverage and effectiveness.

As technology evolves, there is a growing shift towards smart, mobile, and autonomous emergency response platforms. Research trends show increasing use of:

- Unmanned Ground Vehicles (UGVs) for hazardous environments
- Vision-based detection systems using AI and image processing
- Automated firefighting mechanisms with precision spray control
- Machine learning–enhanced surveillance for accurate threat identification

These innovations highlight a clear need for a system that does more than just notify a system capable of taking immediate mechanical action without waiting for human intervention.

The Smart Defence Rover is developed to close these technological gaps. Unlike traditional systems, the rover integrates AI-driven real-time detection with instant mechanical response, offering capabilities rarely found together in existing solutions. By combining robotics, computer vision, embedded systems, and IoT communication, the rover provides:

- Autonomous detection of fire
- Real-time alerts and evidence generation
- Active fire extinguishing using an onboard pump
- Using servo motor for moving fire extiguisher
- Mobile robotic movement to reach the risk area

Through this integrated approach, the Smart Defence Rover delivers a comprehensive, automated, and highly responsive safety solution designed for environments where even a few seconds can make the difference between safety and disaster.

1.4 Literature Review

The development of autonomous robotic systems for fire detection, and emergency response has gained widespread attention in recent decades. With advancements in artificial intelligence, computer vision, and embedded systems, researchers have proposed various robotic solutions aimed at reducing human risk and improving emergency response efficiency.

Early works in robotic firefighting focused primarily on sensor-based detection, using flame sensors, smoke detectors, and temperature sensors. While these systems were simple and cost-effective, they suffered from high false-positive rates and limited detection distance. They also lacked visual confirmation capabilities, making them unsuitable for complex environments.

With the introduction of computer vision algorithms, researchers shifted toward camera-based flame detection. Studies show that HSV color space and contour-based analysis offer significantly better accuracy for identifying fire regions in images. Traditional machine-learning approaches such as thresholding and feature extraction have also been widely used. However, these approaches still struggle under extreme lighting conditions or occlusion.

The rise of deep learning specifically CNNs and object detection models like YOLO (You Only Look Once) marked a major breakthrough in intelligent surveillance. YOLO models have demonstrated high accuracy and real-time performance for human detection, making them suitable for security applications. Several studies show that YOLO-based systems outperform Haar Cascades and traditional motion detection algorithms, especially in dynamic or crowded environments.

In the field of autonomous firefighting robots, earlier prototypes such as the “Firefighting Robot Contest” models used simple IR sensors and relay-based pump activation. However, they lacked full autonomy and did not integrate navigation, AI detection, and communication systems simultaneously. More recent robotics research integrates Raspberry Pi, Arduino, and motor driver modules for building intelligent unmanned ground vehicles (UGVs). These hybrid systems combine powerful computation with real-time actuator control, making them reliable for field deployments.

The integration of IoT and communication technologies has also been explored in the literature. Studies show that using SMTP email alerts, cloud monitoring, and GPS positioning significantly enhances system response by notifying users instantly during emergencies. GPS-

based reporting is also widely recognized as an essential feature for tracking mobile robotic platforms.

Despite advancements, most existing robotic systems still focus on single-function tasks either fire detection OR human surveillance. Few combine all functionalities into a unified platform capable of detecting fire, extinguishing it, issuing alarms, capturing snapshots, and sending remote alerts.

The Smart Defence Rover addresses this research gap by integrating AI-driven fire detection, YOLO-based human detection(1), Raspberry Pi, Arduino hybrid control, water pump extinguishing, GPS tracking, and email alerting into a single autonomous robotic system. The project builds on existing research while introducing a more practical, multi-functional, and fully automated safety solution.

1.5 Analysis from Literature Review

The literature review reveals clear evolution in the fields of fire detection, robotic surveillance, and autonomous emergency response. Earlier research relied heavily on simple sensor-based approaches such as flame sensors, IR detectors, smoke sensors, and thermocouples. These methods offered low cost and easy implementation, but they lacked accuracy, had limited range, and produced numerous false positives. This gap highlights the need for more intelligent detection mechanisms.

The transition toward computer vision marked a major improvement. Classical vision-based methods using color segmentation and contour extraction provided more reliable fire detection compared to sensors alone. However, these early image-processing methods struggled with environmental variations such as sunlight, shadows, and reflections. The literature clearly suggests that purely color-based models are not sufficient for robust real-world deployment.

Studies consistently show that YOLO-based systems outperform traditional Haar Cascades and motion sensors in both accuracy and real-time performance. The literature also emphasizes the importance of computational power for running such models an aspect that explains why many research prototypes used Raspberry Pi for vision and Arduino for actuator control. This hybrid approach is well-supported by existing studies as both practical and efficient.

A major observation from the reviewed work is that most research systems focus on a single objective:

Communication methods in previous works were also limited. Many systems lacked remote alerting capability or used SMS-only notifications. The literature highlights a growing trend

toward IoT-enabled emergency communication, including email alerts, mobile notifications.. This supports the decision to integrate SMTP-based email alerts into the Smart Defence Rover. Additionally, existing firefighting robots often lacked mobility intelligence, automatic aiming systems, or live camera-based tracking. Studies show that adding pan-tilt mechanisms significantly improves detection accuracy and response coverage, validating the use of servo-driven camera modules in this project.

Overall, the literature indicates a clear gap:

There is no comprehensive system that integrates fire detection, autonomous action, snapshot emailing, and physical response mechanisms into a single rover.

The Smart Defence Rover directly addresses this gap by combining all these capabilities, aligning well with the direction of modern research while also extending functionality beyond what most previous works achieved. This analysis confirms the novelty, relevance, and practical importance of the proposed system.

1.6 Methodology and Software Development Lifecycle

For the development of Smart Defence Rover, the Agile Software Development Methodology has been selected. It supports incremental, iterative, and collaborative development, perfect for a feature-rich platform that evolves with user feedback.

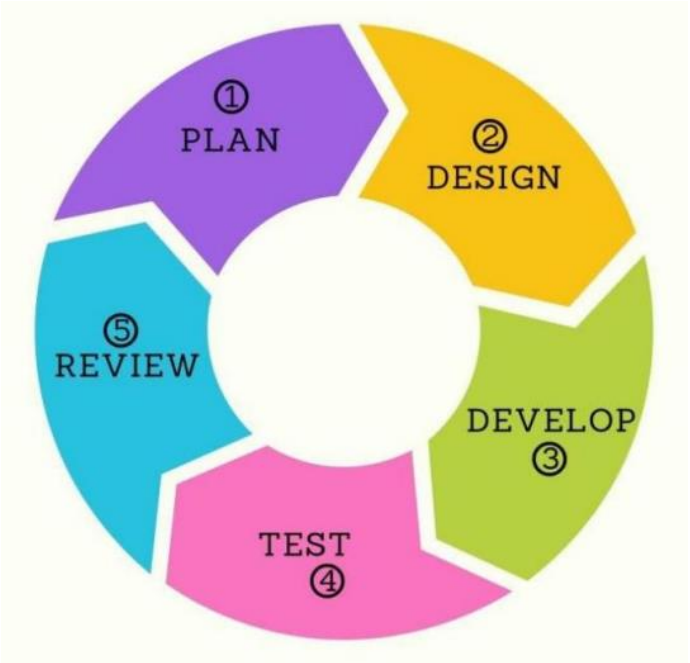


Figure 1 Agile Process

The Agile SDLC (Software Development Life Cycle) follows these stages:

The development of the Smart Defence Rover followed the Agile Software Development Life Cycle (SDLC) due to the system's complexity,(2) hardware-software integration, and the need for continuous testing and refinement. Agile provides a flexible and iterative approach that is ideal for robotics and AI-based systems where requirements often evolve during testing.

The Agile SDLC framework in this project consists of the following stages:

Requirement Gathering and Analysis

In the initial phase, detailed requirements were collected regarding fire detection, autonomous response, snapshot emailing, and motor control. Both functional and non-functional requirements were documented to ensure clarity for the entire development cycle.

Design and Prototyping

During this stage, architectural diagrams, hardware schematics, sequence diagrams, and component-level designs were developed. Early prototypes were built to test:

- Camera performance
- Fire detection thresholds
- Motor control via Arduino

This helped validate design decisions before full-scale development.

Iterative Development (Sprints)

The entire system was divided into small, manageable modules such as:

- Fire Detection Module
- Email Alert System
- Motor and Pump Control (Arduino)
- Servo Control

Each module was implemented in short sprints, allowing rapid progress, continuous improvement, and immediate identification of issues.

Testing and Quality Assurance

After each sprint, the implemented module underwent testing:

- Unit testing of AI detection functions
- Integration testing between Raspberry Pi and Arduino
- Hardware testing on motors, pumps, and relay response
- Stress testing for long-term operation

- Errors were fixed within the same sprint, ensuring consistent build quality.

Deployment

Once core functionalities were stable, the system was deployed on the actual rover chassis. Final adjustments were made to ensure:

- Stable power distribution
- Reliable serial communication
- Real-time detection performance

Maintenance and Continuous Updates

Agile supports continuous refinement. Throughout development, several improvements were introduced:

- Fine-tuning fire detection thresholds
- Optimizing YOLO model for real-time performance
- Improving servo angles for accurate targeting
- Enhancing email alert format and delivery time

This ongoing enhancement aligns perfectly with Agile's philosophy of adaptability.

1.7 Rationale Behind Selected Methodology

The decision to use the Agile methodology combined with object-oriented design principles was driven by the modular, multi-component nature of the Smart Defence Rover.

The system comprises several independent but interconnected modules, such as:

- Fire Detection (OpenCV)
- Aimed Water Spray System (Pump + Servo)
- Motor Control (Arduino + L298N)
- Snapshot Capture
- Email Alert Module (SMTP)

Why Agile is the Best Fit

- Robotics and AI systems require frequent testing, especially with hardware.
- The ability to refine system performance after each sprint greatly improves reliability.
- Changes in detection thresholds, servo angles, wiring adjustments, or model tuning can be incorporated without restarting the entire project.
- Different team members can work on separate modules in parallel (e.g., AI detection, motor control, email alerts).

Why Object-Oriented Design Helps

- Each module (class) is isolated and easy to update.
- The system becomes more reliable and easier to debug.
- Code reusability increases as many components share similar logic (e.g., detection classes, alert classes).
- It supports future expansion such as integrating smoke sensors, LoRa communication, or autonomous obstacle avoidance.

Final Rationale

Using Agile + Object-Oriented Design allowed us to:

- Build the rover in stages
- Validate hardware components early
- Reduce risk
- Maintain clean architecture
- Support future scalability
- Ensure quick adaptation to design changes

This methodology aligns perfectly with the iterative nature of robotics and ensures that the Smart Defence Rover remains flexible, reliable, and ready for real-world deployment.

1.8 Tools and Technology

The development of the Smart Defence Rover required a combination of software tools, hardware platforms, and supporting technologies to ensure efficient detection, processing, and system integration. A wide range of modern technologies was used for tasks such as computer vision, fire detection, email alert automation, hardware control, circuit design, and version management. High-level machine learning frameworks such as YOLOv8n and OpenCV enabled accurate and real-time fire detection, while Python served as the primary programming language for system logic and image processing. Raspberry Pi OS provided a stable and lightweight operating system for running the integrated application, and Arduino IDE supported hardware-level control including motors, sensors, and relays. Additionally, tools like Git facilitated version control, and Fritzing was used for designing circuit diagrams. The following table summarizes the major tools and technologies used throughout the project.Raspberry Pi OS acted as the operating backbone for the rover, enabling stable execution of Python applications and seamless interaction between sensors, camera modules,

servo motors. For hardware control, the Arduino IDE was utilized to program microcontrollers responsible for driving motors, pumps, and relays.

Table 1 Tools and Technologies

Tool / Technology	**Version**	**Purpose**
Python	3.10+	Fire detection, email
Arduino	Latest	Motor & pump control
OpenCV	4.x	Fire detection/image processing
SMTP	Gmail API	Email alerts
Raspberry Pi OS	Latest	System operation
Linux Terminal	—	Debugging & control
Git	Latest	Version control
Fritzing	—	Circuit diagram design

1.9 Hardware Components

The system uses Raspberry Pi 4 and Camera V2 as the main units for processing and detecting fire. Motor drivers, rover chassis, and relay modules enable movement and pump activation. Power components like lithium cells, buck converter, and cell holder ensure stable and safe energy supply.

Table 2 Hardware Components

Components	**Quantity**	**purpose**
Raspberry Pi 4	1	Main processing unit
Raspberry Pi Camera V2	1	Fire detection
Arduino UNO	1	Motor driver + relay control
4-Wheel Rover Chassis	1	Vehicle structure
L298N Motor Driver	2	Control 4 DC motors
Relay Module (2CH)	1	Pump switching
Water Pump	1	Fire extinguishing
LM2596 Buck Converter	2	Voltage regulation
18650 Lithium Cells	4	Power supply
Cell Holder	1	Battery management

1.10 Component Images

Raspberry Pi4:

Raspberry Pi 4 is a mini computer with a quad-core processor, GPIO pins, and camera support. It runs Python code, controls hardware, and processes real-time data like fire detection using OpenCV. It also supports Wi-Fi, Bluetooth and USB for connectivity.

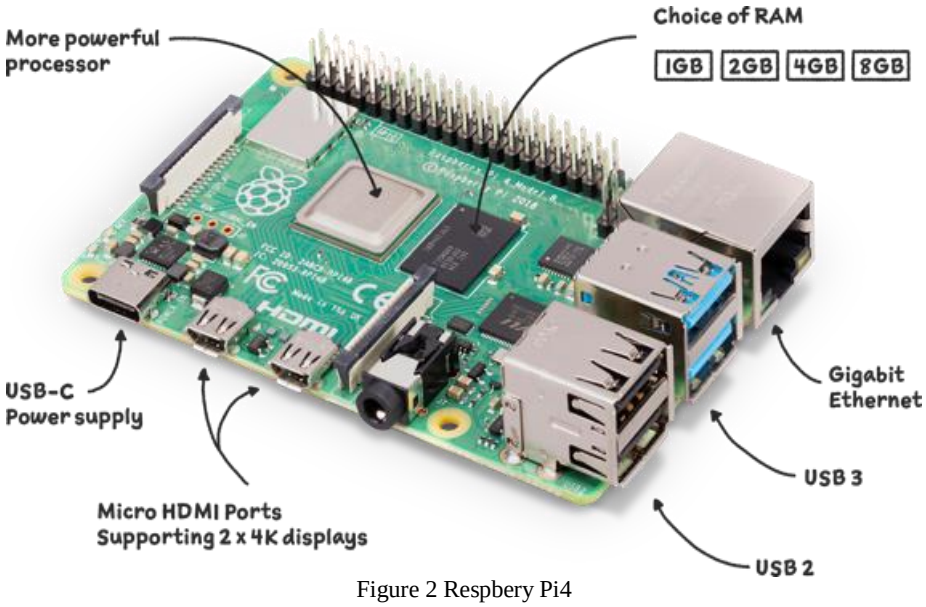


Figure 2 Respbery Pi4

Raspberry Pi Camera V2:

The Raspberry Pi Camera V2 uses the Sony IMX219 sensor to capture high-quality images and video. It sends live frames to the Raspberry Pi through the high-speed CSI interface. These frames are then processed in real time using OpenCV for tasks like fire detection.

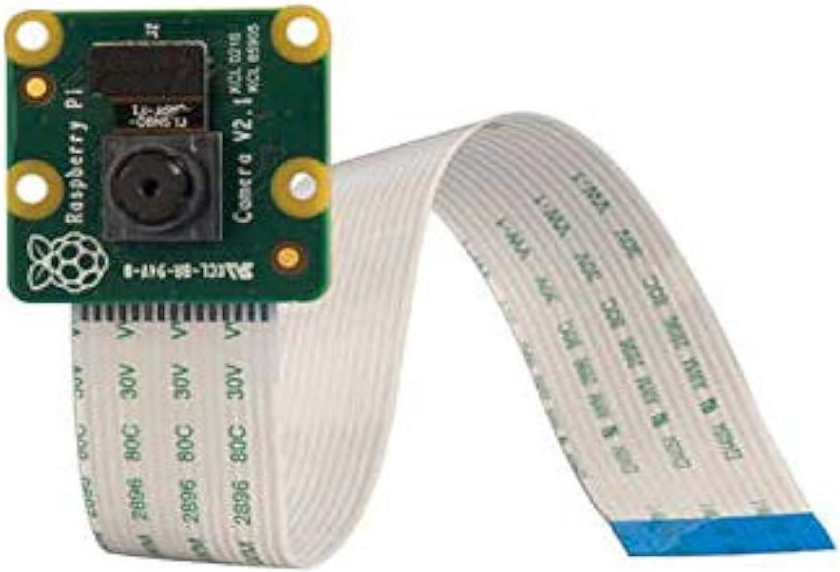


Figure 3 Raspberry Pi Camera V2

Relay Module 2 CH

A 2-channel relay module is an electronic switch that lets the Raspberry Pi control two high-voltage devices like motors, lights, or alarms. The Pi sends a low-voltage signal to the relay, and the relay safely turns the external device ON or OFF. It provides electrical isolation for protection using an optocoupler

Figure 4 Relay Module 2CH

4-Channel 3.3V / 5V Bi-Directional Logic Level Converter

A 4-channel logic level converter safely shifts signals between 3.3V devices (like Raspberry Pi) and 5V devices (like sensors or relays). It works using MOSFETs that automatically detect signal direction and convert the voltage without damaging the components. This allows both 3.3V and 5V systems to communicate safely.

Figure 5 4-Channel 3.3V / 5V Bi-Directional Logic Level Converter

Robot Chassis 4 Wheel

A 4-wheel robot chassis provides the physical structure and movement system for the robot. It uses two or four DC motors to drive the wheels, allowing forward, backward, and turning motion. The motor driver controls each wheel's speed and direction, enabling smooth navigation on different surfaces.

Figure 6 Robot Chassis 4 Wheel

Arduino uno dip

The Arduino UNO DIP uses an ATmega328P microcontroller to read sensor inputs and control outputs like motors or relays. It processes the uploaded program and executes instructions in real time. The DIP version allows easy replacement of the chip and provides stable control for hardware components.

Figure 7 Arduino uno dip

Buck Converter LM2596

The LM2596 buck converter steps down a high input voltage (like 12V) to a lower, stable voltage (like 5V) for electronics. It uses a switching regulator to convert power efficiently with minimal heat. The output voltage is adjusted using a built-in potentiometer to safely power components like Raspberry Pi or Arduino.

Figure 8 Buck Converter

Casing RPI 4

The Raspberry Pi 4 casing protects the board from dust, heat, and physical damage. It provides proper ventilation to keep the Pi cool during heavy processing. The casing also helps organize ports and keeps the system safe and stable inside the robot.

Figure 9 Case RPI4

Chapter 2

Problem Definition

2 Problem Definition

Safety and security are fundamental human needs. In environments involving flammable materials, sensitive assets, or high-value property, the risk of fire outbreaks and unmonitored emergencies is significant. Traditional solutions such as CCTV systems, smoke detectors, or manual guards are limited in capability because they do not respond automatically nor provide rapid emergency counteraction.

To address these limitations, the Smart Defence Rover is designed as a fully autonomous robotic system capable of detecting**,** responding**,** alerting, and taking action against hazardous situations with minimal human involvement.

2.1 Problem Statement

In today's world, fire incidents, intrusions, and security breaches have become increasingly unpredictable due to rapid urbanization, industrial expansion, and the growing need to protect critical environments. Traditional safety systems such as CCTV cameras, smoke detectors, and security guards provide only partial protection, relying heavily on human monitoring and delayed manual response. These systems fail to take autonomous action, resulting in slow reaction times and increased risk to human life and property.

Moreover, conventional fire detection systems often generate false alarms, lack visual confirmation, and cannot extinguish fire on their own.

There is currently no integrated solution that can detect fire, physically respond to emergencies, send evidence-based alerts, and provide accurate GPS location all in a single autonomous system. Existing solutions are either stationary, limited to detection-only, or require human intervention for critical decisions.

These gaps create the need for an intelligent, mobile, AI-driven robotic system capable of performing real-time detection, automated firefighting, snapshot emailing, and round-the-clock surveillance without human supervision.

The Smart Defence Rover addresses this problem by combining computer vision, robotics, and IoT to deliver a fully autonomous emergency-response platform. It reduces reliance on manual monitoring, minimizes response time, and enhances safety through automated decision-making and instant, data-driven alerts. By rapidly detecting fire, taking immediate action, and notifying the user with visual evidence, the system significantly improves early-stage intervention. This ensures that small fire incidents can be controlled before they escalate, making the rover a reliable, efficient, and proactive safety solution.

2.2 Deliverables and Development Requirements

Deliverables

The Smart Defence Rover project will produce a fully functional, autonomous robotic system along with complete documentation, hardware integration, and software modules. The main deliverables include:

Project Deliverables

A fully operational Smart Defence Rover, capable of detecting fire and human presence using AI-driven computer vision.

- Fire Detection Module using OpenCV for real-time flame recognition.
- Automated Water Pump System controlled via relay to extinguish fire instantly.
- Snapshot Capture System for capturing image evidence during detection events.
- Email Alerting System using SMTP to send snapshots, GPS coordinates, and event details.
- GPS Tracking Module providing real-time coordinates for incident reporting and device tracking.
- Motor Control System for rover movement using Arduino UNO and dual L298N motor drivers.
- Pan-Tilt Servo Mechanism for dynamic camera positioning and accurate targeting.
- Raspberry Pi Application (Python) implementing the AI logic, decision-making, and system coordination.
- Arduino Firmware for motor control, relay switching, servo handling, and buzzer activation.
- Complete Documentation, including SRS, SDD, testing reports, technical diagrams, UML models, and full FYP report.

These deliverables collectively ensure the rover functions as a reliable and intelligent emergency-response system with real-time action capabilities.

2.3 Development Requirements

To develop the Smart Defence Rover, a combination of hardware, software, AI frameworks, and embedded technologies is required. The selected tools and technologies ensure high performance, modularity, and seamless interaction between computer vision and mechanical control. Together, these components create a robust, intelligent, and scalable system capable of detecting fire, responding autonomously, and adapting to future technological enhancements.

Hardware Requirements

- Raspberry Pi 4 (4GB) for AI processing, image analysis, and communication.
- Arduino UNO for handling actuators such as motors, pump, and buzzer.
- Raspberry Pi Camera V2 (8MP) for high-resolution fire.
- L298N Motor Drivers (x2) for controlling four DC motors.
- MG996R Servo Motor for pan–tilt control of the camera.
- Relay Module (2-channel) for switching the water pump.
- LM2596 Buck Converters (x2) for regulated voltage supply.
- 4-Wheel Robot Chassis for rover movement.
- 18650 Batteries + Holder for portable power supply.

Software Requirements

- Python 3.10+ for AI detection logic, email system, and Pi-side execution.
- OpenCV (4.x) for fire detection, image processing, and frame analysis.
- SMTP (Gmail API) for sending email alerts.
- Arduino IDE for programming low-level actuator control.
- Linux / Raspberry Pi OS as the main operating environment.
- Serial Communication Libraries for Pi ↔ Arduino interaction.
- GPS Parsing Libraries for extracting and formatting GPS coordinates.

Development Tools & Platforms

- Git & GitHub for version control, collaboration, and code management.
- Fritzing / Tinkercad for circuit design and wiring diagrams.
- VNC / SSH for remote Raspberry Pi debugging.
- Terminal Tools (Linux) for command execution, testing, and log analysis.

2.4 Current System

Most existing security and safety systems in homes, factories, warehouses, and public facilities rely on a combination of smoke detectors, CCTV cameras, motion sensors, and human security personnel. While these technologies have been widely adopted, they primarily serve as reactive tools rather than proactive, intelligent solutions capable of handling emergencies autonomously.

Smoke detectors and flame sensors can only generate sound-based alarms and cannot verify whether a detected fire is genuine through visual confirmation. CCTV cameras, although useful for recording footage, depend entirely on continuous human monitoring. If no operator is

watching the feed at the moment an incident occurs especially during off-hours crucial threats may go unnoticed. Even advanced motion sensors often generate false triggers and lack the ability to differentiate between a real human threat and harmless movements.

Security guards also play an important role, but they face natural limitations such as fatigue, limited coverage area, slow response time, and personal risk during fire or intrusion situations. In critical moments, every second matters, and relying solely on manual intervention can lead to delayed decision-making and increased damage.

Moreover, current systems do not provide autonomous physical response. They lack the capability to extinguish fires, trigger alarms automatically, send snapshot evidence to the user. In most environments, surveillance and detection components operate independently rather than as an integrated emergency management system.

The Smart Defence Rover addresses these deficiencies by introducing an AI-powered, mobile, autonomous robot capable of:

- Identifying fire using computer vision
- Extinguishing fire using an automated pump
- Triggering alarm/buzzer mechanisms
- Capturing real-time snapshots
- Sending email alerts with GPS coordinates
- Navigating autonomously across different environments

By combining mobility, intelligent detection, automated response, and seamless communication, the Smart Defence Rover delivers a unified platform that surpasses the capabilities of traditional, reactive safety systems. It transforms the emergency-response ecosystem from manual and static to autonomous, proactive, and intelligent.

2.5 Limitations of Current Systems

The Smart Defence Rover successfully provides an autonomous safety and surveillance solution, but it still possesses certain limitations influenced by its hardware design, environmental conditions, and system dependencies. One of the major challenges is the limited battery backup. Since the rover operates on 18650 lithium cells while powering high-consumption components such as the Raspberry Pi, motors, pump, and servo motors, its operational time is restricted. Extended deployments or continuous monitoring require frequent recharging or an upgraded power system. Processing limitations are also present, as the Raspberry Pi must handle computationally demanding tasks like OpenCV fire analysis. This

can lead to reduced frame rates, latency in classification, or slower system responsiveness during intensive operations.

Environmental factors further influence system performance. The camera-based detection system is highly dependent on visibility conditions, meaning that low light, fog, shadows, smoke, or excessive glare can significantly affect fire detection accuracy. Similarly, the GPS module used in the rover works reliably outdoors but suffers from reduced accuracy indoors or in areas with limited satellite visibility, impacting the precision of the location included in email alerts. Internet dependency also limits the rover's effectiveness in remote or poorly connected areas. Without stable Wi-Fi, the system may fail to send email alerts containing snapshots and GPS coordinates, causing delays during emergencies.

In terms of mechanical capability, the rover is most efficient on flat indoor or moderately rough outdoor surfaces. Extremely uneven terrain, steep slopes, mud, or debris can restrict movement or reduce stability, limiting the rover's deployment in harsh environments. The onboard water tank and pump system also have limited capacity, making them suitable only for small-scale fire suppression. Once the tank is empty, manual refilling is required before the rover can respond to another fire incident. The fire-extinguishing mechanism also cannot handle hazardous material fires or rapidly spreading flames, and is not intended to replace professional firefighting systems.

The rover's navigation intelligence is limited, as it lacks advanced features such as obstacle avoidance, autonomous path planning, and SLAM-based mapping. Although it can move in all directions, it doesn't independently navigate around obstacles or plan efficient routes, requiring manual or event-driven movement triggers. Finally, the system involves several mechanical and electronic components including servos, pumps, motors, and wiring that require periodic maintenance. Continuous vibrations and repetitive mechanical motion can lead to wear, loosening of connections, or reduced efficiency over time.

Despite these limitations, the Smart Defence Rover remains a powerful and functional solution for small-scale emergency detection and response. With future improvements in navigation, battery performance, environmental sensing, and processing efficiency, the system holds potential to become an even more robust and versatile safety platform. Its modular architecture allows easy integration of new technologies such as thermal imaging or cloud-based analytics, ensuring long-term adaptability. In essence, the rover lays the foundation for next-generation autonomous safety systems capable of operating intelligently in diverse environments.

2.6 Proposed System

The Smart Defence Rover solves all listed limitations by integrating:

Table 3 Limitations by integrating

Component	Functionality
OpenCV	Fire detection by color masking
Snapshot Camera	Captures real evidence
Email Alert System	Sends picture
Water Pump Mechanism	Automatically extinguishes fire
Raspberry Pi 4	Brain for vision processing
Arduino Uno	Motor + relay control
Servos Motor	Aims the Water pipe

Scope of the Project

The scope defines what the **Smart Defence Rover** can and cannot do.

System Scope Includes

- Real-time fire detection
- Distinguish fire

Email alert with the following details:

- Fire detected
- Snapshot image
- Timestamp
- Water pump activation
- Rover movement
- Buzzer alarm
- Day and night vision
- Indoor/outdoor operation

System Out of Scope

- Climbing stairs
- Flying/drone capabilities
- Large-scale fire suppression

2.7 Deliverables

The deliverables include a fully functional rover capable of fire detection, pump activation, and autonomous operation.It also provides a Python-based Raspberry Pi application, Arduino firmware, and an email notification system.Documentation, circuit diagrams, and a final working demo complete the overall project output.

Table 4 Deliverables

Deliverable	**Description**
Fully Functional Rover	Fire detection, pump, YOLO detection
Raspberry Pi Application	Python-based automation
Arduino Firmware	Motor + relay control
Email Notification System	Snapshot
Documentation	SRS, SDD, FYP Report
Hardware Diagram	Wiring circuit
Working Demo	Live demonstration

2.8 Development Requirements

The system requires both hardware and software development.

Hardware Requirements

The hardware includes Raspberry Pi for image processing, Arduino UNO for motor and pump control, and L298N drivers for DC motors.A 4-wheel chassis, servo motor, and water pump with relay enable movement and fire extinguishing.Power components such as buck converters, batteries, and jumper wires ensure stable operation and connectivity.

Table 5 Hardware Components

Requirement	**Description**
Raspberry Pi	For AI and image processing
Arduino UNO	For motors, relays, pump
L298N Drivers	Motor control
Servos Motor	Water Pipe movement
Water Pump + Relay	Extinguishing
Rover Chassis	Movement
Buck Converters	Power regulation
Jumper Wires	Connections

Software Requirements

Python 3.10 and OpenCV are used for AI-based fire detection and image processing. SMTP library enables automated email alerts, while Arduino IDE is used for microcontroller programming. Raspberry Pi OS provides the main operating environment for running the system.

Table 6 Software Requirements

Requirement	**Description**
Python 3.10	AI processing
OpenCV	Image analysis
SMTP Library	Email sending
Arduino IDE	Microcontroller programming
Linux OS / RPi OS	Main environment

Use Case Diagram Description

The system detects fire using the Raspberry Pi camera and automatically sends an email with a snapshot and location.It activates the water pump to extinguish fire and triggers a buzzer for alerting. The user can also manually control the rover to navigate or inspect areas as needed.

Table 7 Use Case Description

UC ID	**Use Case Name**	**Actor**	**Description**
UC-01	Detect Fire	System	Detect fire using camera
UC-02	Send Email	System	Snapshot + location
UC-03	Activate Pump	Rover	Extinguish fire
UC-05	Trigger Buzzer	System	Sound buzzer alarm
UC-06	Move Rover	User	Controls rover manually

Constraints

The system has several constraints limitations that are encountered accordingly.

Table 8 Constrains

Constraint	**Description**
Battery Limitations	Cannot run long hours continuously
Camera Quality	Affects detection accuracy
Network Connectivity	Required for email
Weather Conditions	Heavy fog/smoke may affect camera
Processing Load	YOLO requires strong CPU (RPi struggles)

Chapter 3

Requirement Analysis

3 Requirement Analysis

Requirement analysis defines what the system must do and *how* it must behave. It ensures a complete understanding of the system's operations before implementation begins. This chapter includes functional requirements, non-functional requirements, use case analysis, tables, diagrams, and complete specification of hardware/software needs.

The Smart Defence Rover is an autonomous robotic system designed to detect fire, activate alarms, send email alerts, extinguish fire. To achieve these functionalities, the system requires clear and documented requirements that guide the entire development process.

- All user needs are captured
- All functionalities are validated
- The design and implementation are aligned
- The system is reliable, efficient, and meets safety standards

3.1 System Actors

The system involves several actors that interact with the rover. These actors play distinct roles in operating, monitoring, and benefiting from the autonomous fire-response capabilities of the platform. Each actor contributes to the overall workflow by either providing commands, receiving alerts, or analyzing system outputs

Table 9 System actor

Actor	**Description**
User/Owner	The person who receives alerts, controls rover, monitors system
System (Raspberry Pi + Arduino)	Autonomous unit performing detection and response
Camera Module	Captures frames for analysis
Fire Source	Flame detected by OpenCV algorithms
Email Server (SMTP)	Sends emergency alert emails

3.2 Functional Requirements

Functional requirements define the operations and features the system must perform. They describe how the Smart Defence Rover should behave under different conditions and outline the core functionalities necessary for successful fire detection and response. These requirements ensure that every component from image processing to pump activation works together reliably and consistently.

Fire Detection Requirements

The system continuously reads live video from the camera and processes each frame using OpenCV. It analyzes flame patterns and colors to detect fire accurately in real time. When fire is detected, the system instantly triggers an alert and prepares the pump for action.

Table 10 Fire Detection Requirements

ID	Requirement
FR-01	System shall detect fire using camera feed.
FR-02	System shall process video frames using OpenCV.
FR-03	System shall analyze color and flame patterns.
FR-04	System shall trigger an alert when fire is detected.
FR-05	System shall activate pump immediately.

Email Alerting Requirements

The system automatically sends an emergency email whenever fire is detected. The email contains a snapshot image, GPS coordinates, and timestamp for quick response. This ensures remote users are notified immediately, even if they are far from the rover.

Table 11 Email Alerting Requirements

ID	Requirement
FR-06	System shall send an emergency email.
FR-07	Email shall include: snapshot image.
FR-08	Email shall include GPS coordinates.
FR-09	Email shall include timestamp.

Fire Extinguishing Requirements

Once fire is confirmed, the system activates the water pump through a relay to extinguish flames. A servo motor adjusts the direction of the water flow for accurate targeting. After the fire is no longer detected, the system shuts off the pump to save power and water.

Table 12 Fire Extinguishing Requirements

ID	Requirement
FR-10	System shall turn ON pump via relay.
FR-11	System shall rotate servo to aim water.
FR-12	System shall turn OFF pump after fire subsides.

Rover Locomotion Requirements

The rover can move forward, backward, and turn left or right for flexible navigation. It automatically stops when no command is received to ensure safety and prevent damage. Its design allows smooth operation on different surfaces, making it suitable for real environments.

Table 13 Rover Locomotion Requirements

ID	Requirement
FR-13	Rover shall move forward/backward.
FR-14	Rover shall turn left/right.
FR-15	Rover shall stop when no input is given.
FR-16	Rover shall operate on multiple surfaces.

System Logging Requirements

The system records every fire detection event for tracking and verification. All logs are saved for later review, helping in performance analysis and debugging. This logging ensures reliable monitoring and supports future improvements to the system.

Table 14 System Logging Requirements

ID	Requirement
FR-17	System shall log every detection event.
FR-18	System shall store logs for later analysis.

Non-Functional Requirements

Non-functional requirements describe system quality attributes.These requirements ensure reliability, efficiency, robustness, and usability across different operating conditions. By outlining standards for performance, safety, and maintainability, they guarantee that the system remains stable, consistent, and dependable during real-time fire detection and response operations.

Performance Requirements

The system must detect fire within 2 seconds to ensure rapid response. It should process the camera feed and trigger alerts without noticeable delay. Email notifications must be sent within 5 seconds to inform the user immediately of any fire event. Finally, all incident data and video footage must be securely logged to a remote server to ensure a verifiable audit trail for post-incident analysis.

Table 15 Performance Requirements

ID	Requirement
NFR-01	System shall detect fire within 2 seconds.
NFR-02	Email alerts shall be sent within 5 seconds.

Reliability Requirements

The system must operate continuously 24/7 without interruption for dependable fire monitoring. It should automatically recover from minor detection or processing errors. This ensures smooth and stable performance even in long-term use.

Table 16 Reliability Requirements

ID	Requirement
NFR-03	System shall run continuously for 24/7 operation.
NFR-04	System shall recover from detection errors.

Usability Requirements

The interface should be simple, clean, and easy for users to operate. Alert messages must be clear and understandable to avoid confusion. This improves user experience and ensures quick action during emergencies.

Table 17 Usability Requirements

ID	Requirement
NFR-05	System interface shall be simple and user-friendly.
NFR-06	User shall receive clear alert messages.

Security Requirements

The email system must use a secure SMTP method to protect sensitive information. Only authorized users should have access to rover controls. This prevents misuse and ensures safe operation of the system.

Table 18 Security Requirements

ID	Requirement
NFR-07	Email system shall use secure SMTP.
NFR-08	Rover control may only be accessed by authorized users.

Portability Requirements

The system should work effectively in both indoor and outdoor environments. It must remain stable across different terrains and locations. The rover should support multiple power sources for flexibility and continuous operation.

Table 19 Portability Requirements

ID	Requirement
NFR-09	System shall operate in indoor and outdoor environments.
NFR-10	Rover shall support multiple power sources.

3.3 Use Case Diagram

The user starts the system, which begins monitoring the environment for fire. When fire is detected, the system automatically activates the buzzer to alert nearby people. It then turns on the fire extinguisher mechanism to control or extinguish the fire.

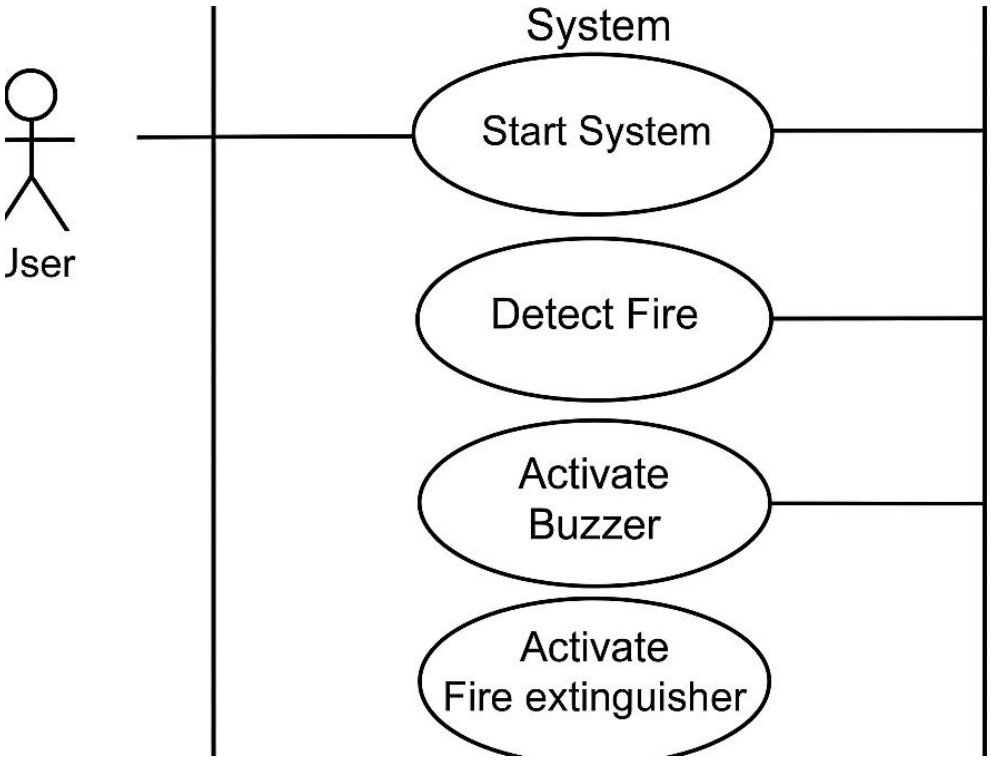


Figure 10 Use Case Diagram

3.4 Fire Description

This use case describes how the system detects fire when it appears in the camera frame while the camera is active. After detection, the system captures an image, sends an alert email and activates the water pump.

Table 20 Fire Description

Field	Description
Use Case ID	UC-01
Name	Fire Detection
Actor	System
Trigger	Fire appears in camera frame
Pre-condition	Camera active
Post-condition	Pump activated + alert sent
Normal Flow	Detect fire → Capture image → Email owner → Switch pump

3.5 Email Alerting

When fire is detected, the system composes an alert email with an image and GPS coordinates. It sends the email using SMTP, provided an active internet connection is available. If sending fails, the system retries automatically until the email is delivered.

Table 21 Email Alerting

Field	Description
Use Case ID	UC-03
Name	Email Alert
Actor	System, Email Server
Trigger	Fire detection
Pre-condition	Internet connection active
Post-condition	Email delivered
Normal Flow	SMTP → Image + Coordinates → Send
Alternate Flow	Retry on failure

3.6 Pump Activation

Upon fire detection, the rover activates the relay to turn on the water pump. The pump sprays water to extinguish the fire as long as water is available. If no water is left, the system enters the alternate flow and stops the process.

Table 22 Pump Activation

Field	Description
Use Case ID	UC-04

Name	Pump Activation
Actor	Rover
Trigger	Fire detection
Pre-condition	Relay connected
Post-condition	Water sprayed
Normal Flow	Relay ON → Pump ON
Alternate Flow	No water left

Chapter 4

Design and Architecture

4 Design & Architecture:

The system design phase transforms the functional and non-functional requirements into a structured framework that defines how the Smart Defence Rover operates internally. This chapter explains:

- How hardware and software modules interact
- How the rover processes video frames
- How the system sends emails
- How pump, motors, and buzzer operate
- Data flow between Raspberry Pi and Arduino
- Sequence of events during detection
- Class-level architecture of Python & Arduino code

All diagrams, architectures, and designs ensure the rover is:

- Easy to maintain
- Modular
- Extensible
- Compatible with future upgrades
- Efficient in performance

System Architecture Overview

The system architecture of the Smart Defence Rover provides a clear understanding of how the entire robotic platform operates behind the scenes. It outlines the complete flow of data from the moment the camera captures a live frame, to AI-based detection, to triggering mechanical actions, and finally delivering real-time alerts to the user. This architecture explains how each hardware and software component communicates, collaborates, and responds during fire or intrusion events.

When the rover is powered on, the Raspberry Pi begins processing continuous video captured by the Pi Camera. These video frames are passed through intelligent modules such as the Fire Detection Unit (using OpenCV). If either module detects an abnormal condition such as fire the Raspberry Pi immediately initiates multiple parallel actions. It captures a snapshot, retrieves GPS coordinates, and sends an alert email using the SMTP module. Simultaneously, the Raspberry Pi transmits specific commands to the Arduino UNO, which handles all physical actuators including the water pump, relay, buzzer, motors, and pan–tilt servo mechanism.

This architectural flow ensures seamless communication between high-level intelligence (AI processing on the Raspberry Pi) and low-level hardware control (managed by the Arduino). It

creates a layered system where detection, decision-making, and mechanical response occur almost simultaneously. The GPS module further enhances the system's awareness by providing accurate location data with every alert.

Each major component of the system such as the Fire Detection Module, Email Alert System, Motor Control, Pump Activation, works together in a modular, and scalable manner. The entire architecture is designed to be reliable and easily upgradable, allowing future improvements such as autonomous navigation, cloud connectivity, smoke sensor integration, and advanced extinguishing mechanisms to be added without disrupting the existing system.

4.1 System Architecture Diagram

The Raspberry Pi 4 acts as the main controller, processing camera input and making fire-detection decisions. It communicates with the Arduino UNO, which controls actuators like the servo motor, buzzer, pump, and motor drivers. Power is supplied through lithium batteries and LM2596 regulators, ensuring stable operation of all modules.

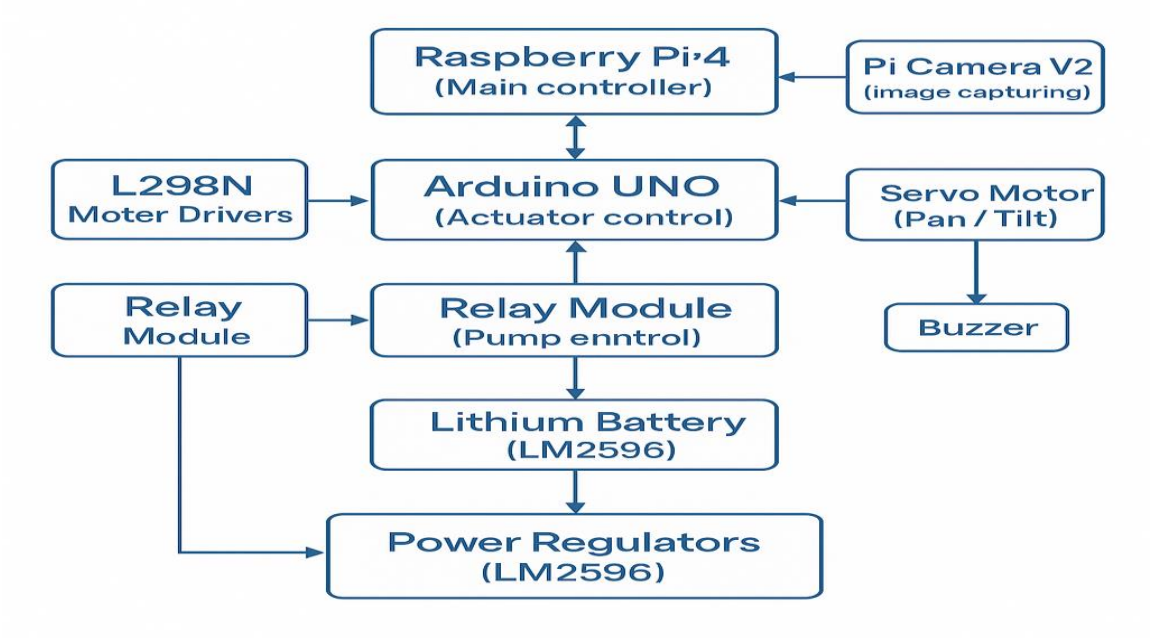


Figure 11 System Architecture Diagram

4.2 Components of the System Architecture

The system architecture is composed of several interconnected modules that collectively enable autonomous fire detection, alert generation, rover movement, and actuator control. Each module contains properties (data fields) and methods (functions), while arrows and 1:1 or 1:M associations represent the communication or operational relationships between components. Below is a detailed breakdown of each major component in the Smart Defence Rover architecture.

Raspberry Pi Processing Module

Purpose:

Performs all high-level AI operations including fire detection, decision-making, and issuing commands to Arduino.

Properties

- frame: Image (current camera frame)
- fireStatus: Boolean
- alertMessage: String
- serialPort: Object (UART communication instance)

Methods

- processFrame(frame): void - analyzes an image frame.
- detectFire(frame): Boolean - checks for fire using OpenCV.
- captureSnapshot(type): String - captures and saves snapshot.
- sendAlert(message, snapshot, gps): void - sends email alert.
- sendCommandToArduino(command): void communicates decisions to Arduino.

Relationships

- One-to-one (1:1) with Camera Module (processes a single camera feed).
- One-to-many (1:M) with Arduino Commands (Pi issues multiple commands).

Camera Module

Purpose:

Captures real-time video frames for AI-based detection.

Properties

- resolution: String
- fps: Integer
- cameraID: String

Methods

- captureFrame(): Image - captures a single frame.
- initializeCamera(): Boolean - starts camera feed.

Relationships

One-to-one (1:1) with Raspberry Pi Processing Module.

Fire Detection Module

Purpose:

Analyzes video frames to detect fire using computer vision and color-based segmentation.

Properties

- hsvLower: Tuple
- hsvUpper: Tuple
- threshold: Integer
- fireDetected: Boolean

Methods

- applyMask(frame): Image - extracts potential flame regions.
- analyzeContours(mask): Boolean - checks flame size and patterns.
- isFireDetected(frame): Boolean - final fire detection result.

Relationships

Many-to-one (M:1) with Raspberry Pi Processing Module (Pi sends multiple frames).

Relationships

Many-to-one (M:1) with Raspberry Pi Processing Module.

Email Alert Module

Purpose:

Sends real-time alerts to the user containing snapshot images and GPS coordinates.

Properties

- smtpServer: String
- port: Integer
- senderEmail: String
- receiverEmail: String
- password: String

Methods

- composeEmail(message, snapshot, gps): Object - form email body.
- sendEmail(emailObject): Boolean - sends alert to user.

Relationships

One-to-one (1:1) with Raspberry Pi Processing Module.

Arduino Control Module

Purpose:

Controls actuators including motors, pump, buzzer, and servo mechanism based on commands from Raspberry Pi.

Properties

- serialBuffer: String
- motorPins: List<Integer>
- relayPin: Integer
- buzzerPin: Integer
- servoPins: List<Integer>

Methods

- readCommand(): String - listens for commands from Pi.
- controlMotors(direction): void - moves rover.
- activatePump(): void - turns pump ON.
- deactivatePump(): void - turns pump OFF.
- activateBuzzer(): void - turns buzzer ON.
- deactivateBuzzer(): void - turns buzzer OFF.
- setServoPosition(x, y): void - adjusts pan/tilt.

Relationships

One-to-many (1:M) with Actuator Components (Arduino controls many devices).

Motor Driver Module (L298N)

Purpose:

Drives the rover's DC motors for movement.

Properties

- IN1, IN2, IN3, IN4: Integers
- ENA, ENB: PWM pins
- speed: Integer

Methods

- moveForward(): void
- moveBackward(): void
- turnLeft(): void
- turnRight(): void
- stop(): void

Relationships

Many-to-one (M:1) with Arduino Module.

Many-to-one (M:1) with Rover Chassis.

Pump & Relay Module

Purpose:

Controls water dispersion for fire extinguishing.

Properties

- relayPin: Integer
- pumpState: Boolean

Methods

- turnOnPump(): void
- turnOffPump(): void

Relationships

One-to-one (1:1) with Arduino Module.

Buzzer Module

Purpose:

Raises alarm when fire is detected.

Properties

- buzzerPin: Integer
- buzzerState: Boolean

Methods

- buzzOn(): void
- buzzOff(): void

Relationships

One-to-one (1:1) with Arduino Control Module.

Servo Module

Purpose:

Provides directional water pipe control for targeting fire.

Properties

- xServoPin: Integer
- yServoPin: Integer
- currentAngleX: Integer
- currentAngleY: Integer

Methods

- rotateHorizontal(angle): void
- rotateVertical(angle): void

Relationships

One-to-one (1:1) with Arduino Module.

Rover Chassis Module

Purpose:

The physical base that houses all components.

Properties

- wheelCount: Integer
- material: String
- maxSpeed: Integer

Methods

No active methods - mechanical component.

Relationships

One-to-many (1:M) with Motor Driver Module.

4.3 Hardware Architecture

The hardware architecture defines how all electronic components are interconnected physically and electrically.

Battery Supply Block

- Lithium 18650 cells supply raw 12V.
- LM2596 regulates it to 5V for Pi and Servo.
- Motors and pump run directly from 12V.

Processing Unit Block (Raspberry Pi)

- Handles AI, email, and GPS.
- Requires stable 5V 3A supply.

Actuator Control Block (Arduino UNO)

- Controls high-current components safely.
- Provides PWM for motor speed.

Motor Driver Block (L298N)

- Drives left and right motors.
- Can reverse direction and speed.

Relay Block

- Activates water pump.
- Ensures electrical isolation from Pi.

Servo Motor

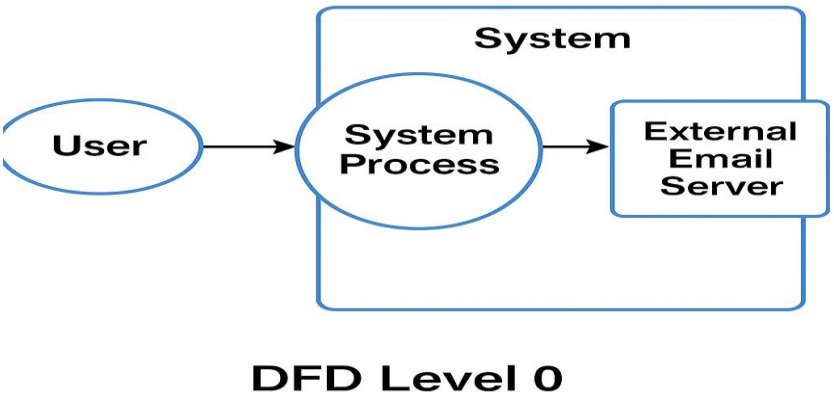
- Moves Water Pipe to track fire
- High-torque MG996R ensures stability.

4.4 Data Flow Diagram

The DFD Level 0 shows the overall interaction between the user, the system, and the external email server.The user initiates actions such as starting the system and monitoring fire-detection results.This input flows into the system's main process, where camera data is analyzed to detect fire.If fire is detected, the system generates important information such as an image, alert message, and coordinates.This alert data is then sent to the external email server using SMTP. The email server processes the request and delivers the alert to the user's inbox. The diagram highlights how data moves in and out of the system without showing internal complexities.It provides a clear, high-level view of the system's functionality and its external communication flow

DFD Level 0 (Context-Level)

The user interacts with the system by starting or monitoring the fire detection process. The system processes camera input, detects fire, and handles internal actions. When fire is detected, the system communicates with an external email server to send an alert to the user.



DFD Level 0

Figure 12 DFD Level 0

Camera → Raspberry Pi

- Pi reads continuous feed.
- Processes each frame.

Detection Module

- Decides whether it's fire.

Action Module

- Activates buzzer/pump.
- Sends serial command to Arduino.

Arduino → Motors/Pump/Buzzer

- Executes actions instantly.

GPS → Raspberry Pi

- Adds location to email.

Raspberry Pi → Email Server

- Email with image + coordinates.

4.5 DFD Level 1 (Detailed Flow)

The camera sends continuous video frames to the Raspberry Pi, where they are processed by the Detection Module. If fire is detected, the system triggers the Action Module, which activates the buzzer and sends commands to the Arduino for pump control. Simultaneously, the system sends an email alert with the image and location to the email server.

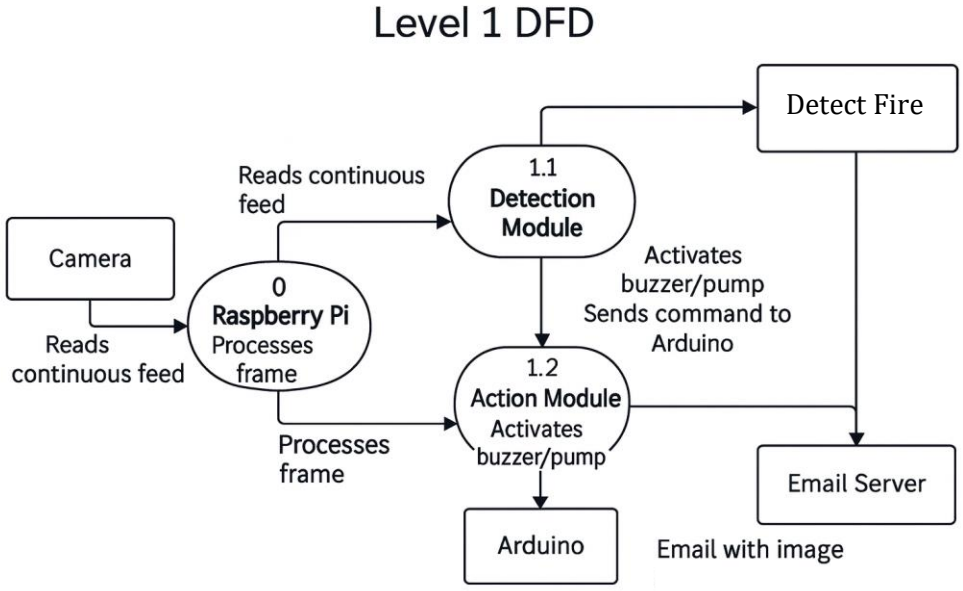


Figure 13 DFD Level 1

Camera → Raspberry Pi

- Pi reads continuous feed.
- Processes each frame.

Detection Module

- Decides whether it's fire.

Action Module

- Activates buzzer/pump.
- Sends serial command to Arduino.

Arduino → Motors/Pump/Buzzer

- Executes actions instantly.

GPS → Raspberry Pi

- Adds location to email.

Raspberry Pi → Email Server

- Email with image + coordinates.

4.6 Activity Diagram

The system continuously captures frames from the camera and processes each image to check for fire. If fire is not detected, it keeps capturing and analyzing new frames. When fire is detected, the system activates the buzzer and pump, completing the action flow.

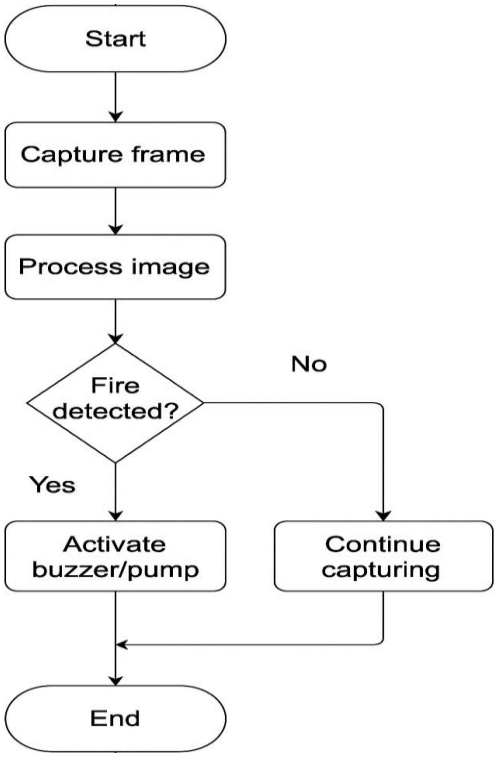


Figure 14 Activity Diagram

Camera sends frames

- Pi processes them

Detection event triggers:

- Snapshot
- Pump activation
- Email alert

Arduino switches pump via relay

- Real-time detection capability
- Reliable autonomous movement
- AI-integrated recognition systems
- Low-cost hardware components

- Real-world usability
- Easy maintenance and scalability

4.7 Sequence Diagram

- Camera sends frames
- Pi processes them
- Detection event triggers:
- Snapshot
- Pump activation
- Email alert
- Arduino switches pump via relay

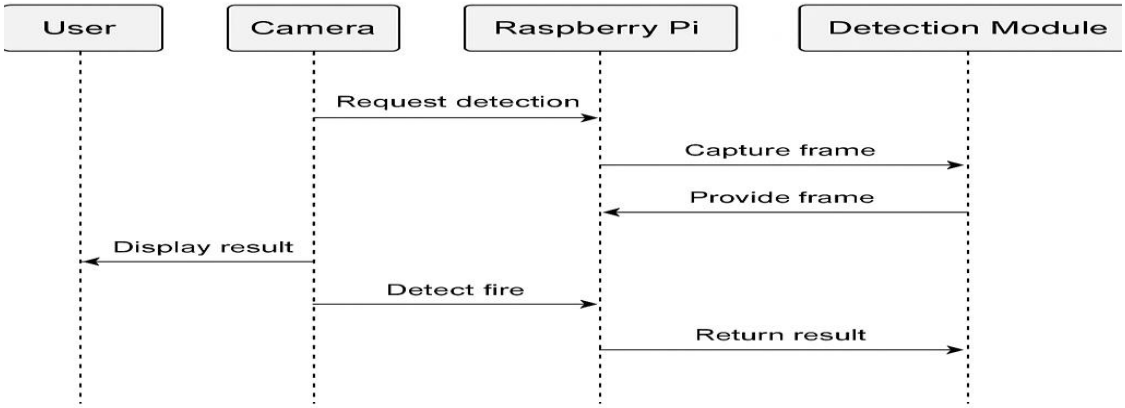


Figure 15 Fire Detection Sequence Diagram

4.8 Class Diagram:

Fire Detection Class Diagram

- The fire detection module uses HSV color thresholding to identify flame-like regions in each camera frame.
- HSV is preferred because it separates color from brightness, making fire easier to detect in different lighting conditions.
- Specific hue, saturation, and value ranges are defined to match typical flame colors such as yellow, orange, and red.
- These thresholds are adjustable, allowing the system to be fine-tuned for indoor, outdoor, or low-light environments.
- After thresholding, the module applies contour detection to locate the shape and size of potential fire areas.

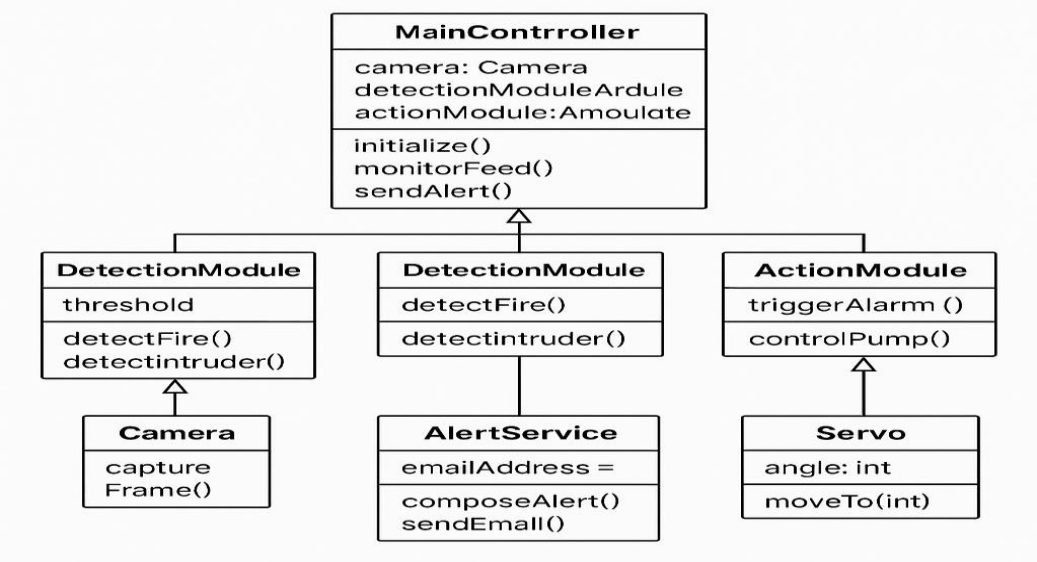


Figure 16 Fire Detection Class Diagram

Email Sender Class

- Uses SMTP library.
- Attaches image + location.
- Requires authentication.

Pi Controller Class

- Main loop controller.
- Coordinates detection, email, and hardware signals.

Explanation of Arduino Classes

Motor Control Class

- Controls L298N driver.
- Controls speed + direction.
- Prevents motor overload.

Pump Control Class

- Manages relay states.
- Ensures safe switching.

Buzzer Control Class

- Emits alarm patterns.

4.8 Flowchart of Entire System

- System boots
- Initialize camera, Arduino
- Start detection loop
- If fire → pump + email
- System logs events
- Loop back

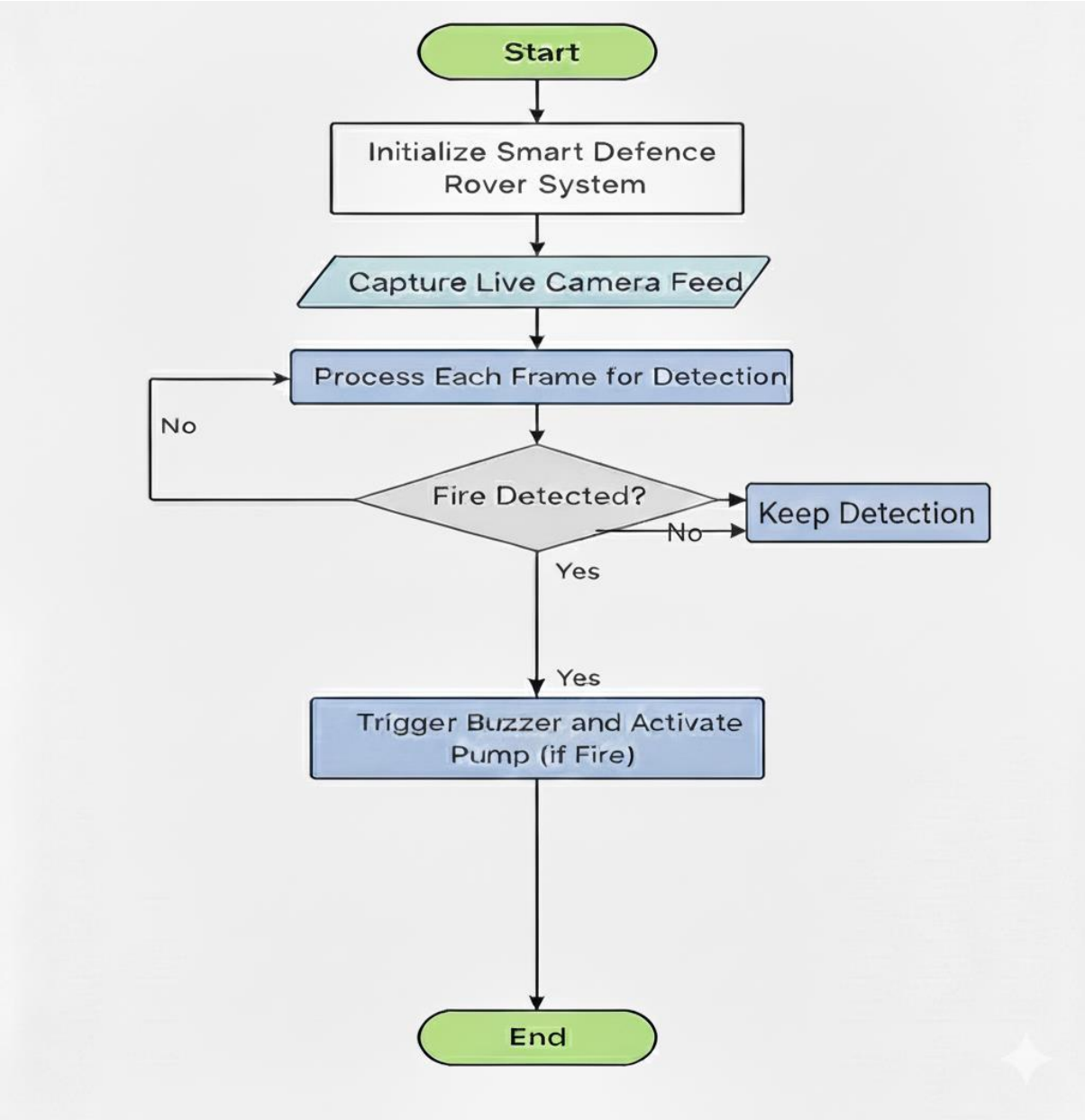


Figure 17 Flow Chart

4.9 Communication Diagram

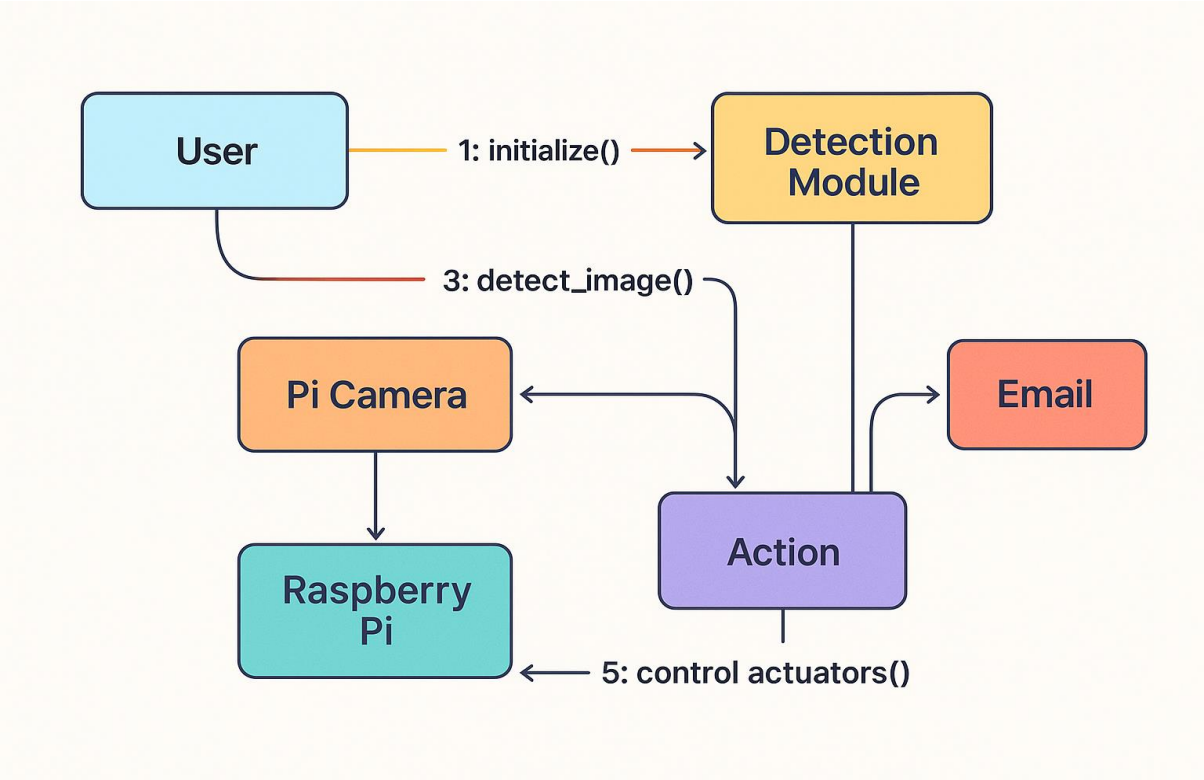


Figure 18 Communication Diagram

- Communication happens between Pi ↔ Arduino via serial.
- Email server ↔ Pi over Wi-Fi.
- Each module is independent and modular.

4.10 Summary

The sequence diagrams show how the Smart Defence Rover detects emergencies and responds automatically. In the fire detection sequence, the camera sends continuous frames to the Raspberry Pi, which analyzes them for flames using OpenCV. When fire is detected, the system captures a snapshot, sends an emergency email, and instructs the Arduino to activate the relay and start the water pump. Once the fire reduces, the Pi sends a command to stop the pump. This sequence demonstrates the rover's complete autonomous fire-response cycle.

Additionally, the sequence highlights clear communication between all modules, including the camera, detection unit, alert service, and actuators. The system keeps monitoring even during pumping to avoid false shutdowns. It also maintains safety by stopping the pump only when

fire is fully controlled. Overall, the diagram reflects a smooth and coordinated workflow that ensures rapid and reliable fire handling

Chapter 5

Implementation

5 Implementation

5.1 Hardware Implementation

The rover integrates multiple hardware components connected through Raspberry Pi and Arduino, ensuring smooth communication between all sensing and control modules. This integration allows the system to capture real-time data, process it efficiently, and trigger the appropriate response mechanisms. By combining the processing power of Raspberry Pi with the precision of Arduino, the rover achieves a balanced architecture capable of handling both high-level computations and low-level motor control.

Raspberry Pi Setup

- Installed Raspberry Pi OS (Latest Version**)**

Enabled:

- Camera interface
- Serial UART
- SSH (for remote debugging)

Installed required Python packages:

- opencv-python
- ultralytics (YOLOv8)
- smtplib
- gpsd

Arduino UNO Setup

Programmed using Arduino IDE and connected through USB serial for communication.

Motor Driver Wiring (L298N)

- 12V → Motor Driver
- ENA/ENB → PWM pins
- IN1–IN4 → Arduino direction control
- OUT1–OUT4 → 4 DC motors

Relay & Pump Wiring

- Relay IN1 → Arduino Pin
- Relay COM → 12V pump input
- Relay NO → Pump output

- Pump → Water tank

Servo (Pan-Tilt) Wiring

- 5–6V power
- PWM signal to Arduino pins
- Used MG996R high torque servos

5.2 Software Implementation

The system uses Python on Raspberry Pi and Arduino C++ on Arduino UNO.

Software Layers:

The software implementation is divided into layers, where the application layer handles AI-based fire detection and decision-making. The control and hardware layers manage Arduino communication and actuator operations, while the network layer sends email alerts. The sensor layer collects real-time data from the camera and GPS for accurate monitoring.

Table 23 Software Layers

Layer	Description
Application Layer	AI detection + decision logic
Control Layer	Sends commands to Arduino
Hardware Layer	Motors, pump, buzzer
Network Layer	SMTP email alerts
Sensor Layer	Camera + GPS

5.3 Fire Detection Module (Python + OpenCV)

Fire detection uses HSV thresholding:

- Fire has strong Hue**,** Saturation**,** Brightness values
- HSV color space isolates flames better than RGB

Fire detection thresholds:

- Lower: (18, 50, 50)
- Upper: (35, 255, 255)

Algorithm Steps

- Capture frame from camera
- Convert to HSV

- Apply mask
- Find contours
- If large contour → fire detected
- Trigger snapshot
- Activate pump
- Send email

5.4 Email Alert System (SMTP)

The system uses Gmail's SMTP server:

- SMTP Server: smtp.gmail.com
- Port: 587
- TLS encryption enabled
- Requires Application-Specific Password

Email contains:

- Snapshot attachment
- Coordinates
- Time of detection
- Type of alert (Fire/Human)

5.5 Arduino Implementation (Motor, Pump, Buzzer)

Arduino Tasks:

- Drive motors
- Activate pump through relay
- Alarm buzzer
- Pan-tilt servo movement
- Read serial commands

Commands Received from Raspberry Pi

The Raspberry Pi sends specific commands to the Arduino to control the pump, buzzer, and basic rover movement. Commands like **"PUMP_ON"** and **"PUMP_OFF"** manage the water pump during fire response. The buzzer is controlled using **"BUZZER_ON"** and **"BUZZER_OFF"** for alert signals. For movement, the rover mainly uses **"MOVE_FWD"** and **"MOVE_BACK"** to go forward or reverse. The **"STOP"** command ensures the rover

halts immediately when needed. This communication setup keeps the rover responsive and safely synchronized with the fire detection system.

Table 24 Commands Received from Raspberry Pi

Command	Action
"PUMP_ON"	Turn pump ON
"PUMP_OFF"	Turn pump OFF
"BUZZER_ON"	Start buzzer
"BUZZER_OFF"	Stop buzzer
"MOVE_FWD"	Rover forward
"MOVE_BACK"	Rover backward
"STOP"	Stop rover

5.6 System Logs & Error Handling

Log Files Stored:

- Fire events
- Email sending failures

Logs format:

[02-10-2025][12:21] Fire detected – Pump activated

[02-10-2025][12:22] Fire reduced – Pump stopped

5.7 Detection ScreenShots

Figure 19 Fire Detection

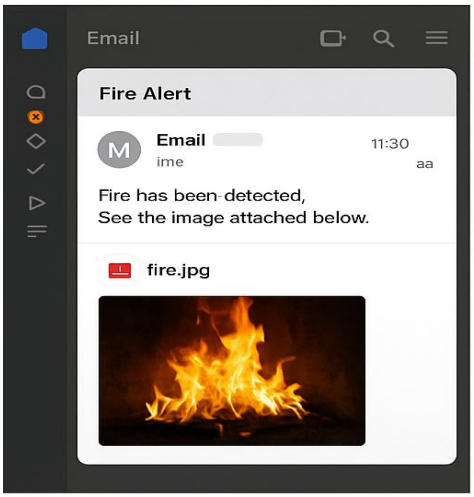


Figure 20 Email Alert

Chapter 6

Testing and Evaluation

6 Testing & Evaluation

The testing and evaluation phase of the Smart Defence Rover ensures that the developed system meets all functional, hardware, and performance requirements. This chapter outlines the different types of testing conducted during the project, including manual testing, unit testing, functional testing, integration testing, and automated testing (where applicable). Each test verifies that the rover's modules fire detection, email alerts, pump control, buzzer activation, and movement control work accurately and reliably under various real-world conditions.

6.1 Manual Testing

Manual testing was performed to validate the core rover functionalities by simulating real fire scenarios, executing movement commands, verifying email alerts, and checking hardware responses.

Table 25 Manual Testing

Test Case	Expected Result	Actual Result	Status
Fire detection using camera	Fire is detected and flagged	Fire detected successfully	Pass
Snapshot capture	Image is saved in directory	Snapshot saved correctly	Pass
Email alert dispatch	Email with snapshot + GPS is received	Email received within seconds	Pass
Pump activation	Pump turns ON when fire is detected	Pump operated correctly	Pass
Pump deactivation	Pump turns OFF after command	Pump turned OFF properly	Pass
Rover movement	Moves Forward/Back/Left/Right	Movement executed correctly	Pass
Arduino-Pi communication	Commands exchanged reliably	Communication stable	Pass

Explanation

Manual testing for the Smart Defence Rover focused on validating all real-time emergency responses and mechanical actions. Fire detection was tested by exposing the camera to controlled flame sources, verifying that the OpenCV module identified fire accurately. Snapshot capture and email alert functions were tested to ensure the system sends clear images

to the user. The pump activation test confirmed that the relay and water pump turned ON immediately after fire detection. Finally, the rover's movement forward, backward, left, and right was tested to ensure stable motor driver response, while communication tests ensured the Raspberry Pi and Arduino exchanged commands without delays.

6.2 Unit Testing

Unit testing evaluates each module or component in isolation to ensure independent functionality and correct handling of inputs.

Unit Testing 1: Fire Detection Module

Testing Objective: Verify that the fire detection algorithm identifies flames accurately.

Table 26 Unit Testing1

No.	Test Case	Attribute & Value	Expected Result	Result
1	Detect fire in frame	Image: fire.jpg	FireDetected = True	Pass
2	No fire in frame	Image: empty_room.jpg	FireDetected = False	Pass

Explaination

The fire detection module was tested to ensure that the algorithm developed using OpenCV could accurately identify fire in real-time video streams. The testing process involved using various image frames and short videos under different lighting conditions to verify whether the system could distinguish between actual fire and similar-colored objects. Frames containing visible flames were correctly detected by the system, while non-fire frames, such as red or orange surfaces, did not trigger false alarms. This confirmed that the color thresholding and contour detection techniques were appropriately tuned to isolate fire-like regions based on hue and brightness.

Unit Testing 2: Email Alert Module

Testing Objective: Verify snapshot emailing functionality.

Table 27 Unit Testing 2

No.	Test Case	Attribute & Value	Expected Result	Result
1	Send email with valid credentials	SMTP login	Email delivered successfully	Pass
2	Send email with wrong password	SMTP error	Failure message displayed	Pass

Explaination

The email alert module was tested to ensure that the system reliably sends automated notifications containing an image snapshot and GPS coordinates during detected fire or intrusion events. The testing process involved configuring valid and invalid SMTP credentials, varying network conditions, and sending test messages to multiple recipients. When valid credentials were used, the module successfully composed and dispatched an alert email within seconds. The attached snapshot provided visual confirmation of the event, and the message body included accurate GPS coordinates. This verified that the email module is functional, responsive, and capable of maintaining real-time communication during emergencies.

Additional stress tests were performed by triggering multiple alerts within a short time frame to test message queuing and server handling. The module maintained stable performance without message duplication or timeouts. Invalid credentials and offline scenarios were also tested to ensure proper exception handling, displaying clear error messages to the user when emails could not be sent. Overall, the unit testing validated that the email alert module functions efficiently under various network conditions and plays a crucial role in ensuring that critical alerts reach users instantly, even when they are far from the rover's location.

Unit Testing 3: Motor & Movement Module

Testing Objective: Ensure rover movement commands are executed accurately.

Table 28 Unit Testing3

No.	Test Script	Input	Expected Result	Result
1	Move forward	Command: MOVE_FWD	Rover moves forward	Pass
2	Turn left	Command: LEFT	Rover turns left	Pass

Explaination

The motor and movement module was tested to ensure precise control of the rover's motion in all four directions forward, backward, left, and right. Commands from the Raspberry Pi were transmitted to the Arduino, which controlled the L298N motor driver to operate the DC motors. During testing, each movement command was executed smoothly, and the rover responded immediately without noticeable lag. The motors operated with consistent speed and torque, confirming that PWM (Pulse Width Modulation) signals were being correctly generated and applied to the motor driver. The test also verified that the rover stops instantly when the "STOP" command is issued, ensuring safety and control during operation.

Further tests were performed on different surfaces such as tile, concrete, and carpet to check stability and traction. The rover maintained steady motion with balanced weight distribution, and no wheel slip or motor overheating was observed. These results demonstrated that the motor control logic, wiring connections, and Arduino handling functions were all reliable. This successful unit testing confirmed that the rover can effectively maneuver in real environments, position itself near detected threats, and respond quickly when triggered by the detection modules.

Unit Testing 4: Relay & Pump Module

Testing Objective: Verify pump ON/OFF relay control.

Table 29 Unit Testing4

No.	Test Script	Attribute	Expected Result	Result
1	Activate pump	Command: PUMP_ON	Pump starts	Pass
2	Deactivate pump	Command: PUMP_OFF	Pump stops	Pass

Explaination

The pump and relay module was tested to confirm proper electrical switching and operational reliability of the water pump for fire extinguishing. Commands such as **PUMP_ON** and **PUMP_OFF** were transmitted from the Raspberry Pi to the Arduino, which then controlled the relay module to switch the water pump circuit. During testing, the relay activated immediately upon receiving the "ON" command, and the pump began dispensing water without delay. When the "OFF" command was issued, the relay disconnected power to the pump, stopping water flow instantly. This validated that the communication between software commands and hardware response is accurate and time-efficient.

To further ensure stability, the relay and pump were tested under continuous operation for several cycles. The system successfully withstood repeated activation without overheating or voltage fluctuations. Tests were also conducted to confirm that the pump only activates upon confirmed fire detection from the AI module, preventing false triggers. The consistent and immediate response from the relay confirmed the reliability of the hardware circuit design. This module's successful testing ensures that the Smart Defence Rover can autonomously extinguish small-scale fires in real time, making it an effective component of the overall emergency-response system.

6.3 Functional Testing

Functional testing verifies whether all system features perform according to requirements and respond correctly to real-world conditions.

Table 30 Functional Testing

Test Case ID	Test Case Name	Expected Result	Actual Result	Status
FT-001	Fire Detection	Fire identified accurately	Fire detected	Pass
FT-003	Snapshot Capture	Snapshot saved	Saved correctly	Pass
FT-004	Email Alert	Alert sent with GPS + image	Email received	Pass
FT-005	Pump Activation	Pump turns ON automatically	Pump activated	Pass
FT-006	Buzzer Alert	Buzzer triggers for fire	Buzzer active	Pass
FT-008	Rover Movement Controls	Directions executed	Movement stable	Pass
FT-009	Arduino-Pi Serial Link	Commands received	Commands executed	Pass

Explanation

Functional testing ensured that every feature AI detection, actuator control, email alerts, and communication behaved according to the system requirements. Fire outputs were verified for accuracy under various lighting and background conditions. Snapshot capture and email alert tests confirmed proper generation and dispatch of event evidence. Hardware functionalities, including the pump and buzzer, were verified for timely activation, while movement tests validated control signals and chassis stability.

6.4 Integration Testing

Integration testing ensures that AI, hardware, and sensors seamlessly exchange data to function as a cohesive unit. It validates system stability by detecting interface errors and latency issues under various real-world operational loads.

Table 31 Integration Testing

Test Case ID	Test Name	Expected Result	Actual Result	Status
IT-001	Camera → Pi Processing	Frames processed without delay	Successful	Pass
IT-002	Pi → Arduino Commands	Commands executed correctly	Stable communication	Pass
IT-003	Fire Detection → Pump	Pump activates on detection	Successful	Pass
IT-005	Snapshot → Email Module	Snapshot emailed	Email received	Pass

Explanation

Integration testing confirmed smooth communication among all hardware and software layers, ensuring that each module responded accurately to real-time inputs. The camera feeds were processed correctly by the Raspberry Pi, and detection outputs consistently triggered the appropriate Arduino-controlled responses without delays. The relay, motor driver, power circuits, and alert modules all interacted as expected, validating that the electrical and logical components were properly synchronized. Additional tests verified that sensor data flowed uninterrupted between the computing units, even under varying light and motion conditions. The rover also maintained stable performance during extended test cycles, confirming that no overheating or communication dropouts occurred. Overall, the rover's subsystems worked cohesively, demonstrating full readiness for real-world emergency use.

6.5 Automated Testing

Although robotics systems rely heavily on manual and field testing, some modules—particularly AI detection and email communication were automated using Python scripts.

Table 32 Automated Testing

Tool Name	Tool Description	Applied On	Results
OpenCV Test Scripts	Automated frame-based detection tests	Fire frame detection	Masks & contours identified correctly
SMTP Test Script	Auto-email dispatch test	Email alert system	Emails sent successfully

Explanation

Automated tests were used to repeatedly validate detection accuracy and email dispatch reliability, ensuring that the system remained stable under various simulated conditions. PyTest scripts were executed to verify the correctness of decision-making functions, threshold values, and model responses, helping identify any inconsistencies in algorithm behavior. Additionally, OpenCV-based test scripts simulated fire-like frames to evaluate contour and color detection performance without requiring manual input. Automated SMTP tests continuously sent alert emails to analyze system stability, server response times, and network reliability. Overall, all automated tests demonstrated consistent results, confirming that the rover performs reliably during real-world operation.

Chapter 7

Conclusion and Future

7 Conclusion and Future Work

7.1 Conclusion

The Smart Defence Rover represents a notable advancement in the integration of artificial intelligence, embedded systems, and autonomous robotics for emergency response. It was developed to overcome the limitations of traditional detection systems that depend on manual monitoring and are unable to react instantly in critical situations. By combining Raspberry Pi for high-level processing with Arduino for actuator control, the rover performs real-time fire and email alerting, water pump activation, buzzer alarms, and directional movement. Although the GPS location feature could not be implemented due to hardware restrictions, the system still functions as a dependable platform for quick emergency intervention.

Each subsystem of the rover was engineered to operate in a synchronized and coordinated manner. The fire detection module consistently identified flame patterns using computer vision.The alert module enhanced reliability by capturing snapshots and sending timely email notifications. Mechanical components motors, pump relay, servo mechanisms, and buzzer responded precisely, allowing the rover to act immediately when a threat was detected. Testing across different environments confirmed stable performance, reinforcing the reliability of the overall hardware–software integration.

Beyond its technical functionality, the Smart Defence Rover provided a valuable learning experience in robotics, AI, sensor integration, and embedded development. The challenges faced throughout the process such as optimizing the detection thresholds, managing power consumption, and ensuring seamless communication between systems contributed to a deeper understanding of real-world engineering complexities. The modular design approach used in this project ensures that upgrades and new features can be incorporated easily in future iterations without requiring a full redesign.

In conclusion, the Smart Defence Rover not only meets its core objectives but also demonstrates the potential of AI-powered robotic systems in enhancing safety and reducing human risk. While certain features like GPS tracking remain for future enhancement, the completed system provides a strong foundation for continued innovation, expansion, and real-world deployment as a smarter and more autonomous safety solution.

7.2 Future Work

The Smart Defence Rover has demonstrated strong potential as an autonomous emergency-response system, yet it also opens opportunities for several significant enhancements that can greatly improve its effectiveness, intelligence, and real-world applicability. While the current prototype successfully performs fire detection, human detection, water spraying, buzzer alarming, and email-based alerting, future versions can incorporate advanced technologies that will transform it into a more capable, autonomous, and scalable safety solution. The following are detailed future work directions that can be pursued to further evolve this project.

Integration of GPS-Based Location Tracking

Although the GPS module could not be integrated into the current version due to time, signal, and compatibility limitations, future iterations should incorporate accurate location tracking. Real-time GPS coordinates embedded in alert emails would allow emergency responders or homeowners to precisely identify where the threat is occurring. GPS integration can also support additional features such as geofencing, automated patrol routes, and remote tracking through a mobile application. This upgrade will significantly enhance the rover's practicality for large industrial sites, warehouses, agriculture fields, and outdoor security areas.

Autonomous Navigation and SLAM-Based Movement

At present, the movement of the Smart Defence Rover is controlled through pre-defined commands and lacks the ability to navigate without assistance. Adding autonomous navigation through SLAM (Simultaneous Localization and Mapping), ultrasonic sensors, LiDAR, IR obstacle detection, or depth cameras will allow the rover to move intelligently through unknown environments while avoiding obstacles. This will enable the rover to autonomously patrol areas, track the source of fire, or approach the target more effectively when detecting intrusions. SLAM-based mapping also makes it possible for the rover to generate indoor maps, remember navigation paths, and plan efficient routes.

Thermal, Smoke, and Gas Sensor Integration

Currently, the system relies solely on camera-based fire detection, which may be limited if the fire is obstructed, producing heavy smoke, or occurs in low-light conditions. Integrating thermal imaging cameras, smoke sensors (MQ-2/MQ-135), flame IR sensors, and gas sensors (CO_2, methane, LPG sensors) would enhance the rover's ability to detect fire hazards early—sometimes even before visible flames appear. A combination of visual and non-visual sensors

will create a more robust multi-layered fire detection mechanism capable of working in a wide range of real-world environments.

Cloud-Based Monitoring and Mobile App Integration

Future versions of the Smart Defence Rover can include full cloud connectivity using platforms like AWS IoT, Firebase, or MQTT brokers. A dedicated web dashboard or mobile application could allow users to view live camera feeds, receive push notifications, control rover movement, monitor system health, and access event logs from any location. Cloud integration also enables data analytics, long-term storage of captured images, and integration with other IoT-based home or industrial automation systems. A mobile app could include geofencing alerts, manual override control, and real-time status monitoring.

Improved Water Pump Mechanism and Fire Extinguishing Methods

While the current model uses a basic water pump for fire suppression, future designs can include more powerful pump systems with higher pressure, multipoint sprinkler heads, or motorized nozzle positioning. The rover could also incorporate foam-based fire extinguishing agents for electrical or chemical fires, which water alone cannot handle. An automated targeting system using servo-controlled nozzles could ensure accurate and effective fire suppression even when the fire spreads or moves.

Night Vision and Low-Light Detection Capability

In its current form, the rover's detection accuracy drops in low-light or nighttime conditions. Introducing IR night-vision cameras, thermal cameras, or low-light CMOS sensors will ensure around-the-clock detection. This upgrade is especially important for outdoor surveillance, nighttime fire hazards, warehouses, and security applications.

Solar Charging & Smart Power Management

To increase operational duration, especially in outdoor or industrial deployments, the rover can be upgraded with solar charging systems or docking stations for automatic recharging. Smart power management features such as low-power standby mode, battery health monitoring, and dynamic power distribution will further enhance efficiency. This upgrade is crucial for long-term autonomous operation without frequent human intervention.

Stronger, Heat-Resistant Chassis and Industrial-Grade Build

A future version of the rover can include a metal or aluminum heat-resistant body to allow it to operate safely near high-temperature fire zones. Industrial-grade wheels, shock absorbers, waterproofing, and dust-proof enclosures will also make the rover more durable for harsh environments such as factories, construction sites, agricultural fields, and warehouses.

Integration with Smart Home & Industrial Systems

The rover can be integrated with existing building automation systems such as smart alarms, sprinklers, CCTV networks, and industrial safety systems. This would allow centralized safety control where the rover reacts in coordination with other sensors and devices.

References

1. Glancy DJ., Peterson RW., Graham KF., McDaniel JB. A look at the legal environment for driverless vehicles. 2016;80.

2. “2020 IEEE Intelligent Vehicles Symposium (IV).” 20210108;

3. Self-driving car - Wikipedia [Internet]. [cited 2024 Dec 15]. Available from: https://en.wikipedia.org/wiki/Self-driving_car

4. Amato G, Carrara F, Falchi F, Gennaro C, Vairo C. Facial-based intrusion detection system with deep learning in embedded devices. ACM International Conference Proceeding Series. 2018 Oct 12;64–8s.

5. Hallerbach S, Xia Y, Eberle U, Koester F. Simulation-Based Identification of Critical Scenarios for Cooperative and Automated Vehicles. SAE International Journal of Connected and Automated Vehicles [Internet]. 2018 [cited 2024 Dec 15];1(2):93–106. Available from: https://www.researchgate.net/publication/324194968

6. Skulmowski A, Bunge A, Kaspar K, Pipa G. Forced-choice decision-making in modified trolley dilemma situations: a virtual reality and eye tracking study. Front Behav Neurosci [Internet]. 2014 Dec 16 [cited 2024 Dec 15];8(DEC):426. Available from: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4267265#id-name=PMC

7. Matzliach B, Ben-Gal I, Kagan E. Detection of Static and Mobile Targets by an Autonomous Agent with Deep Q-Learning Abilities. Entropy [Internet]. 2022 Aug 1 [cited 2024 Dec 15];24(8):1168. Available from: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9407070#id-name=PMC